ABB Driver Analysis Copilot

Technical Walkthrough and Manager Q and A Preparation

Purpose: Help business users understand selected drivers, analyze forecast scenarios, ask questions against structured and unstructured knowledge, generate plots, run simulations, and export or email conversations.

Browser UI	->	Python backend APIs in backend/server.py
server.py	->	LangGraph agent in backend/agent.py
Agent	->	Tools: structured SQL, KPI text, plotter, simulator
Tools	->	SQLite database, KPI text file, generated plot images
Conversation history	->	Recent chats, PDF export, email sharing

1. Executive Summary

This application is a local AI copilot for ABB driver analysis. It combines a browser-based frontend, a Python backend, a LangGraph tool-using agent, a local SQLite knowledge base, a KPI definition text file, Plotly chart generation, deterministic simulation logic, PDF export, and SMTP email sharing.

The most important design point is that the LLM is not answering everything from memory. It is connected to tools. Structured numerical questions are answered through SQL over SQLite. Definitions and driver-selection reasoning come from a local KPI knowledge text file. Scenario calculations are handled by deterministic Python logic. Plots are generated from retrieved data.

One-line manager answer: This is a local full-stack AI copilot where the UI talks to a Python backend, the backend invokes a LangGraph GPT agent with bound tools, and the tools ground responses in SQLite data, KPI text, Plotly charts, and deterministic simulation logic.

2. High-Level Architecture

Browser UI	->	Python backend APIs in backend/server.py
server.py	->	LangGraph agent in backend/agent.py
Agent	->	Tools: structured SQL, KPI text, plotter, simulator
Tools	->	SQLite database, KPI text file, generated plot images
Conversation history	->	Recent chats, PDF export, email sharing

Main folders

- frontend/: browser UI files - app.js, index.html, styles.css.
- backend/server.py: backend web server, API routes, conversation storage, PDF, email, transcription.
- backend/agent.py: LangGraph agent, prompts, tool definitions, SQL routing, plotting, simulation.
- data/: SQLite database and KPI definition text.
- assets/: ABB logo and static UI assets.

3. Email Service

The email feature uses SMTP, specifically Gmail SMTP in the current setup. It is not using Gmail API, OAuth, or Google Cloud. The backend reads the email host, port, SSL flag, sender email, and app password from the .env file.

```
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=465
EMAIL_USE_SSL=true
EMAIL_USER=...
EMAIL_PASSWORD=...
PROFILE_NAME=Anisha Mahanty
PROFILE_TITLE=Data Scientist
```

When the user clicks Email, the frontend sends the recipient email and name to the backend. The backend checks that the conversation has messages, generates a PDF from the conversation history, attaches that PDF, prepares the email body and signature, and sends it through SMTP.

Manager answer: The app uses secure Gmail SMTP with an app password. It dynamically generates the conversation PDF, attaches it, and sends it from the configured sender account. It does not use Gmail API in this version.

Likely follow-up questions

- Is this production ready? For a local prototype, yes. For enterprise production, we should use ABB-approved mail infrastructure, Microsoft Graph, or an internal SMTP relay.
- Are passwords in code? No. Secrets are read from .env, which should not be committed.
- Why port 465? SMTP_SSL on port 465 worked reliably on the Windows/ABB network, while STARTTLS on port 587 was unstable during testing.

4. Role Of server.py

server.py is the backend controller for the entire application. It serves the frontend, exposes API routes, stores and retrieves conversation history, calls the LangGraph agent, generates PDFs, sends emails, handles recipients, returns dashboard tile data, and performs audio transcription.

Important routes

- GET / - serves the browser UI.
- GET /api/dashboard/tiles?division=ELSP - dashboard tile data.
- POST /api/chat/stream - streaming chat endpoint used by the UI.
- POST /api/transcribe - mic audio to text.
- GET /api/conversations/<id>/pdf - PDF download.
- POST /api/conversations/<id>/send-email - email conversation as PDF.
- GET/POST/DELETE /api/recipients - saved email recipients.

Manager answer: server.py is the bridge between the browser, the agent, SQLite, PDF generation, email delivery, and transcription.

- Why not Flask or FastAPI? This prototype uses the built-in HTTP server to keep it lightweight. For production, FastAPI would be better for validation, auth, middleware, logging, and deployment.

5. Frontend And Backend Communication

The frontend is a browser-based React-style UI served from frontend/app.js, frontend/index.html, and frontend/styles.css. It communicates with the backend through HTTP APIs using fetch(). The chat endpoint uses streaming so the UI can show live progress status while the agent works.

```
Frontend fetch()
-> POST /api/chat/stream
-> backend/server.py
-> LangGraph agent
-> streamed status events and final answer
```

The UI immediately shows the user message, then listens for streamed events such as assistant started, selected tool, tools returned result, summarization completed, and final parsed response.

Manager answer: The frontend and backend communicate through REST-style HTTP APIs. The chat API streams progress events so users can see what the agent is doing live.

6. Mic And Speech-To-Text

The mic feature records audio in the browser using browser audio APIs. The browser does not do the transcription itself. It records audio, converts it into a base64 payload, sends it to POST /api/transcribe, and the backend sends it to OpenAI transcription. The returned text is inserted into the chat input box.

```
User clicks mic
-> Browser records audio
-> POST /api/transcribe
-> backend sends audio to OpenAI transcription
```

```
-> text returned to frontend
-> text appears in input box
```

Manager answer: The browser records audio; the backend uses OpenAI transcription to convert speech to text. Audio is not intended to be stored.

- Failure cases: browser mic permission denied, unsupported browser recording API, missing OpenAI API key, or network issue.

7. Agent Architecture

The agent is built in backend/agent.py using LangGraph. The assistant is an LLM with bound tools. Based on the user question and tool descriptions, it decides whether to answer directly or call one of the tools.

```
START
-> assistant
-> tools if tool call is needed
-> tool_result_trace
-> auto_plot_check
-> summarization
-> END
```

Main tools

- structured_data_tool: routes questions to SQLite tables and runs generated SQL.
- unstructured_kpi_tool: reads KPI definitions and driver evaluation rationale from a text file.
- plot_tool: generates charts from the latest structured data.
- simulation_tool: deterministic forecast calculation using driver values and elasticities.

Manager answer: The agent is a LangGraph workflow. GPT decides which bound tool to use. The tools ground the response in data, definitions, plots, or deterministic calculations, and the summarization node turns raw outputs into a business-facing answer.

- Is routing hardcoded? Graph routing is deterministic after the LLM emits tool calls, but tool selection is driven by the LLM and tool descriptions.
- Why LangGraph? It gives explicit control over nodes, tool execution, streaming status, summarization, and future extensibility.

8. Structured Data Tool

Structured data is stored in SQLite at data/driver_analysis_chatbot.db. The main tables are Forecast_KB, Driver_Contribution_KB, and Monthly_Data.

The structured tool uses two internal LLM steps. First, it selects the relevant table based on table descriptions. Second, it generates SQL using only the selected table's schema and context. Python then executes the SQL on SQLite and returns rows to the agent.

```
User question
-> select relevant table
-> generate SQL from selected table schema
-> execute SQL on SQLite
-> return rows
-> summarizer creates final answer
```

Manager answer: Structured questions are answered through text-to-SQL over SQLite. The tool first selects the right table, then generates and runs SQL, so numerical answers are grounded in the database rather than free-form model memory.

- SQL is internal and not shown in the UI by default.
- The tool can support questions spanning more than one structured table when needed.

9. Unstructured KPI Tool

The unstructured knowledge is a text file extracted from the KPI definition PDF. It contains KPI meanings, synonyms, business meaning, selected/rejected driver rationale, and evaluation notes. Because the file is small and bounded, the whole text can be passed into the LLM context when the tool is called. There is no vector database or RAG pipeline in this version.

Manager answer: The unstructured tool reads a local KPI and driver-evaluation text file. It is used for definitions and selection reasoning. Since the knowledge base is small, full-text context is simpler and avoids retrieval errors.

- If the knowledge base grows significantly, vector search or document indexing can be added later.

10. Plotting

Plotting is based on structured results. When structured data is retrieved, the backend saves the result to CSV and can generate a Plotly chart. The chart is saved as a PNG under plots/ and served to the frontend through /plots/<filename>.png.

For follow-up requests such as 'plot this' or 'make it a line plot', the assistant can reuse the latest structured data instead of querying SQL again. This reduces latency and prevents accidental changes in the data being plotted.

Manager answer: Charts are generated from retrieved structured data. The plotter creates a PNG image, the backend serves it, and the frontend displays it in the conversation. Follow-up plot requests reuse the latest structured result.

11. Simulator

The simulator is a deterministic Python calculation tool. The LLM decides when to call it and extracts the user's requested inputs, but the actual calculation is Python logic. It uses baseline driver values and elasticities from the structured database.

```
driver_delta = new_driver_value - baseline_driver_value
impact = driver_delta * elasticity
final_forecast = baseline_forecast + sum(driver_impacts)
```

Manager answer: Simulation is not model hallucination. GPT only decides when to call the tool and with what inputs. The forecast impact calculation itself is deterministic Python logic grounded in database values.

12. PDF Generation

PDF generation happens in server.py using ReportLab. Conversations are stored in SQLite in conversations and conversation_messages tables. When the user clicks PDF, the backend fetches the conversation title and messages, lays them out with ABB branding, and returns a downloadable PDF.

```
GET /api/conversations/<conversation_id>/pdf
-> fetch conversation from SQLite
-> create branded ReportLab PDF
-> return application/pdf download
```

Manager answer: The PDF is generated dynamically from stored conversation history. It is not a screenshot. The backend reads messages from SQLite and uses ReportLab to create a branded PDF with topic, user questions, and assistant answers.

- For email, the same PDF generation function is reused, then attached to an SMTP email.

13. Conversation Persistence

The app stores conversations in SQLite so that recent chats, message reload, PDF export, and email sharing work. The recipients list is also persisted in SQLite.

Main persisted concepts

- conversations: id, title, created_at, updated_at.
- conversation_messages: role, content, visible_text, closing_text, plot_urls, timestamp.
- recipients: saved recipient name and email.

Manager answer: Conversation state is persisted locally in SQLite. This lets the app show recent conversations, reload previous messages, generate PDFs, and send conversations by email.

14. End-To-End Message Flow

```
User sends message
-> frontend displays user message immediately
-> POST /api/chat/stream
-> backend creates/loads conversation
-> backend saves user message
-> LangGraph agent runs
-> selected tool executes if needed
-> summarization node creates final answer
-> backend saves assistant message
-> frontend displays final response
```

The status indicator in the UI is powered by streamed backend events. This is why the user can see processing status while the agent is working.

15. Strengths And Production Considerations

Strengths

- Modular separation between frontend, backend, agent, tools, and data.
- Tool-grounded responses instead of relying only on free-form LLM answers.
- Streaming trace makes agent behavior more transparent.
- SQLite keeps local setup simple for a prototype.
- Conversation history can be reused for PDFs, emails, and follow-up plots.

Production considerations

- Move from local HTTP server to FastAPI or another production web framework.
- Use enterprise authentication and role-based access control.
- Move secrets from .env to a managed secrets store.
- Use an approved enterprise mail service instead of Gmail SMTP.
- Use a managed database for multi-user deployment.
- Add logging, monitoring, audit trail, and automated tests.

16. One-Minute Script

This is a local AI driver-analysis copilot. The frontend is a React-style browser UI served by a Python backend. The backend exposes APIs for chat, dashboard tiles, PDF export, email, transcription, and conversation history. The core intelligence is a LangGraph agent using GPT with bound tools. Depending on the question, the agent can query structured SQLite data, read KPI definition text, generate plots, or run a deterministic forecast simulator. Conversations are stored in SQLite, which allows recent chats, PDF generation, and email sharing. Email is sent via Gmail SMTP with an app password, and PDFs are generated dynamically using ReportLab from stored conversation history.